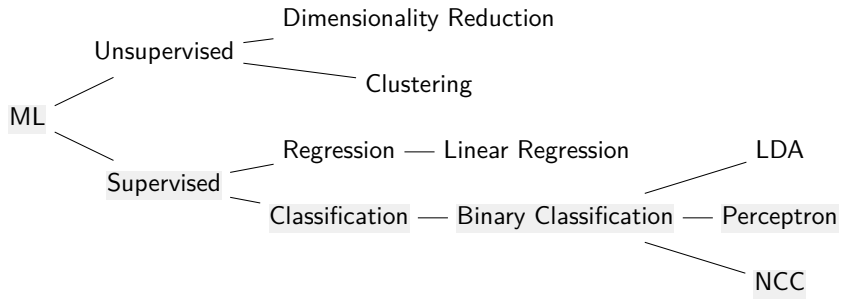
Classification



Binary Classification

Machine Learning Basics

Based on knowledge (dataset) we want to answer or solve a problem.

Notation hint

Here $\mathbf{w}$ and $\mathbf{x}$ (bold letters) denote vectors in $\mathbb{R}^d$. We usually use d for the dimension of a datapoint and n for the number of datapoints.* Consequently, we denote $X \in \mathbb{R}^{n \times d}$ as the matrix containing the datapoints $\mathbf{x}_i$ (i.e a list of datapoints).

$$X = \begin{bmatrix} \mathbf{x}_1 & \mathbf{x}_2 & \dots & \mathbf{x}_n \end{bmatrix}$$

y_i refers to the label corresponding to $\mathbf{x}_i$.

*not always consistent, but in these notes.

Example

We have a dataset comprising of 100 PNG images with a resolution of 8×8 and corresponding labels `dog` and `cat`. Each image could be represented as a 64-dimensional vector $\mathbf{x} \in \mathbb{R}^{64}$, and each label y could be represented by 0 or 1.

Binary Classification

Based on a dataset containing data points with class labels belonging to two classes, we want to predict the class labels of future unlabeled data points. Consider this example of a two-dimensional dataset.

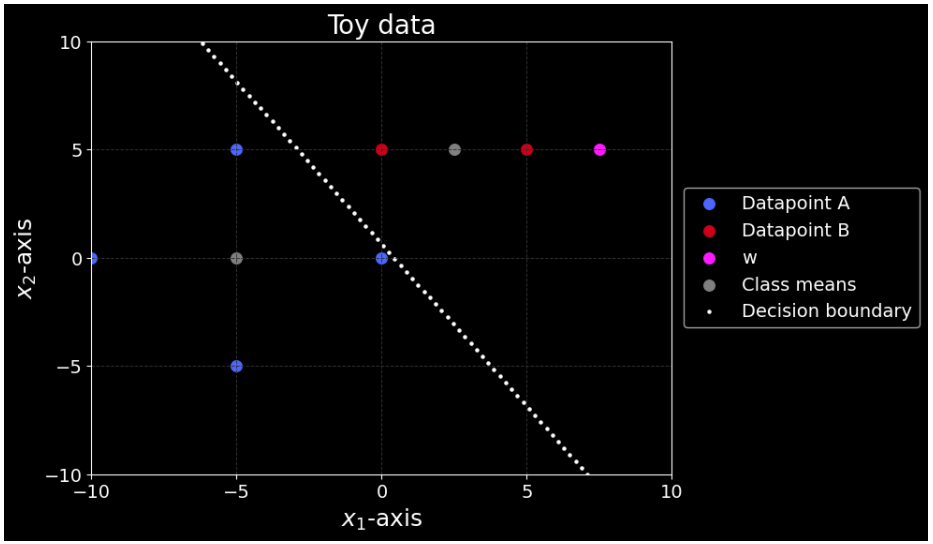


Figure 1: Plot of a two-dimensional dataset containing two classes (Blue A and Red B). The axes represent the first and second entries of each data point. The weight vector $\mathbf{w}$ is shown as a magenta point. The decision boundary is the set of points satisfying $\mathbf{w}^\top \mathbf{x} - \beta = 0$. Below is how we can represent both datasets in Python.

```
A = np.array([[0,0],[-5,5],[-10,0],[-5,-5]])
B = np.array([[0,5],[5,5]])
```

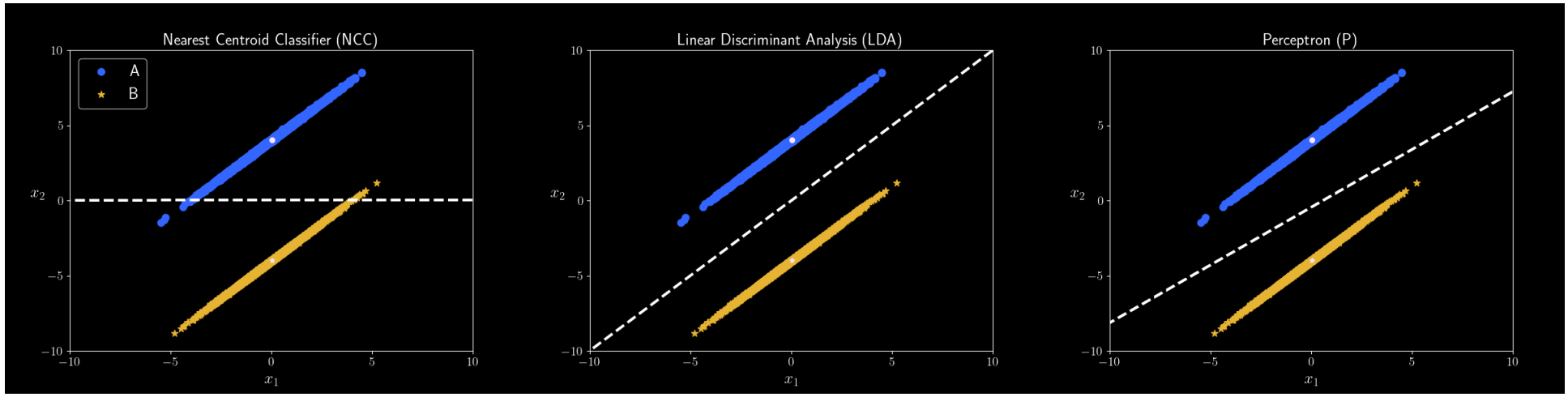


Figure 1: Each plot shows the same dataset containing two-dimensional datapoints $x \in \mathbb{R}^2$ belonging to two classes (blue and orange). The white dashed line in each plot shows the decision boundary which resulted from NCC, LDA (later) or the Perceptron. Note that the perceptron can have multiple outcomes for the same dataset.

In general we can write a binary classification model as

Binary classification model

Let $\mathbf{w}, \mathbf{x} \in \mathbb{R}^n$ and $\beta \in \mathbb{R}$ then

$$sign(\mathbf{w}^\top \mathbf{x} - \beta)$$

returns either -1 or 1.

We call the set of points $\mathbf{x} \in \mathbb{R}^n$ that fullfill the equation $\mathbf{w}^\top \mathbf{x} = 0$ decision boundary.We will learn 3 methods to derive $\mathbf{w}, \mathbf{x}$ and β for binary classification. In this chapter Nearest Centroid Classifier (NCC) and Perceptron. Later, Linear discriminant analyis (LDA). The result of NCC and LDA will always be the same on the same dataset because the result is given by a formular.

Nearest Centroid Classifier

NCC formular for $\mathbf{w}$ and β

Given Dataset with datapoints $\mathbf{x} \in \mathbb{R}^n$, denote x_+, x_- datapoints belonging to 0,1

$$\mathbf{w} = \bar{\mathbf{x}}_- - \bar{\mathbf{x}}_+ \quad \text{and then} \quad \beta = \frac{1}{2}(\mathbf{x}_-^\top \mathbf{x}_- - \mathbf{x}_+^\top \mathbf{x}_+)$$

Example

Let $\begin{bmatrix}0\\0\end{bmatrix}, \begin{bmatrix}0\\2\end{bmatrix} \in \mathbb{R}^2$ be in class - and $\begin{bmatrix}1\\1\end{bmatrix} \in \mathbb{R}^2$ be in class $+$. Then we can calculate $\mathbf{w}$ by

$$\frac{1}{2}\left(\begin{bmatrix}0\\0\end{bmatrix} + \begin{bmatrix}0\\2\end{bmatrix}\right) = \begin{bmatrix}0\\1\end{bmatrix} = \bar{\mathbf{x}}_- \text{ and } \begin{bmatrix}1\\1\end{bmatrix} = \mathbf{x}_+ \text{ then } \mathbf{w} = \begin{bmatrix}-1\\0\end{bmatrix} = \begin{bmatrix}0\\1\end{bmatrix} - \begin{bmatrix}1\\1\end{bmatrix}$$

and β can be calculated by

$$\beta = \frac{1}{2}(1-2) = -\frac{1}{2}$$

Perceptron

Unlike NCC, Perceptron uses an iterative algorithm to update $\mathbf{w}, \beta$ until there are no missclassified points left. For example look at the right side of this page.

Algorithm 1 Perceptron Learning Algorithm

Input: Training data $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, where $\mathbf{x}_i \in \mathbb{R}^d$, $y_i \in \{-1, 1\}$ and step size $\eta > 0$.
Output: Weight vector $\mathbf{w}$, bias β

```
Initialize w ← random ∈ ℝ^d, b ← random ∈ ℝ
repeat
    # Loop through each training example
    for each (x_i, y_i) do
        # Check if x_i is missclassified
        if y_i(w^⊤ x_i + β) ≤ 0 then
            w ← w + η y_i x_i
            β ← β + η y_i
until no misclassifications
```

Because of random initialization of $\mathbf{w}$ and β the perceptron algorithm can result in different decision boundaries.

Task

How can we modify the algorithm to only use modified $\mathbf{x}$ and $\mathbf{w}$?

Solution Augmented Notation

Given $\mathbf{x}, \mathbf{w} \in \mathbb{R}^d$ then $\tilde{\mathbf{x}}, \tilde{\mathbf{w}} \in \mathbb{R}^{d+1}$. We modify the last entry of $\mathbf{x}$ to be 1 and the last entry of $\mathbf{w}$ as β.

$$\tilde{\mathbf{x}}_i^\top \tilde{\mathbf{w}}_i = \tilde{\mathbf{x}}_{i1} \cdot \tilde{\mathbf{w}}_{i1} + \tilde{\mathbf{x}}_{i1} \cdot \tilde{\mathbf{w}}_{i1} + \ldots \tilde{\mathbf{x}}_{id} \cdot \tilde{\mathbf{w}}_{id} + 1 \cdot \beta = \mathbf{x}^\top \mathbf{w} + \beta$$

Perceptron loss function; Perceptron Error

The function

$$\mathcal{L}_{\mathcal{P}} : \mathbb{R}^d \to \mathbb{R}, \quad w \mapsto -\sum_{m \in \mathcal{M}} w^\top x_m y_m$$

is called the **Perceptron loss functions**

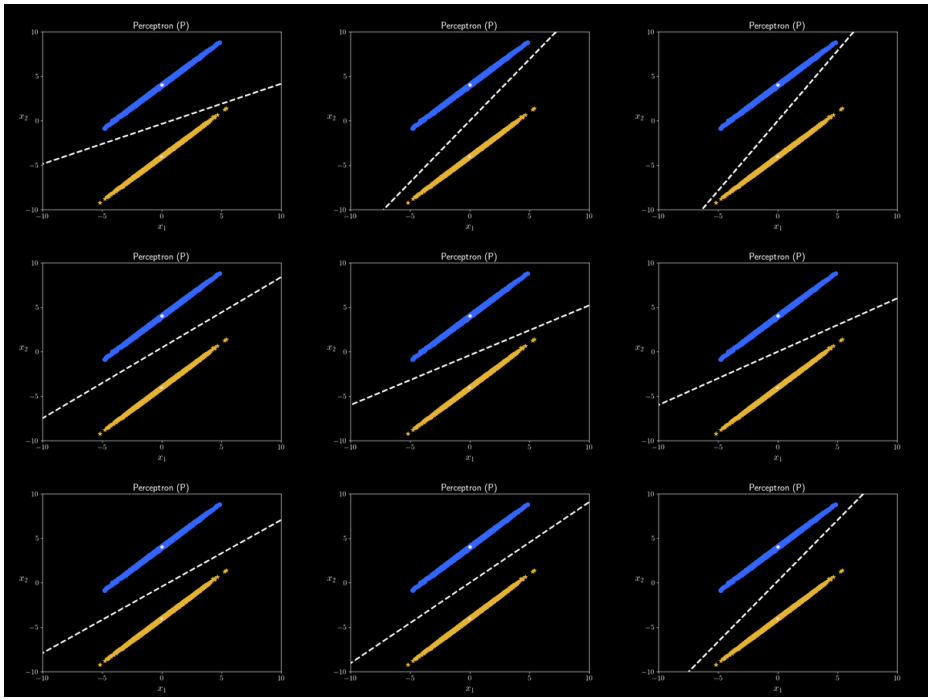


Figure 2: This figure shows the same two-dimensional dataset consisting of two classes (blue and orange) and the result of the perceptron learning algorithm nine times. In each plot a different decision boundary (White dashed line) is depicted. This is a result of a random starting point. Different random initialized $\mathbf{w}$ and β result in a different set of $\mathcal{M}$ (Set of indices of missclassified points).

In lecture 6 we will learn that the idea of the perceptron can be extended by creating a whole network of perceptrons. The perceptron is a building block for the multilayer perceptron (Neural network).

Linear Discriminant Analysis

Data often follows an underlying distribution, and most commonly a Gaussian distribution. Linear Discriminant Analysis tries to exploit this. For more stochastic math background, go to the preliminary math section at the beginning of these notes (Under Construction).

Covariance Matrix

The covariance matrix contains information about the covariance of each possible combination of random variables for a dataset in $\mathbb{R}^{d\times d}$. It is definied as

Covariance Matrix Σ_X

$$\Sigma_X = \begin{bmatrix} Var(X_1) & \dots & Cov(X_1, X_d) \\ \vdots & \ddots & \vdots \\ Cov(X_d, X_1) & \dots & Var(X_d) \end{bmatrix} \in \mathbb{R}^{d\times d}$$

We can we can estimate it numerically by

Empirical Covariance Matrix $\hat{\Sigma}_X$ Formular

$$\hat{\Sigma}_X = \frac{1}{n}(X - \bar{X})(X - \bar{X})^\top$$

where $X \in \mathbb{R}^{d\times n}$ datamatrix containing datapoints in each column, $\bar{X} \in \mathbb{R}^{d\times n}$ containing the mean $\mu \in \mathbb{R}^d$ of all datapoints in each column

$$\bar{X} = \begin{bmatrix} | & | & & | \\ \mu & \mu & \dots & \mu \\ | & | & & | \end{bmatrix}.$$

Note that the formular above can be used for non centered data, since we substract the mean of each datapoint first.

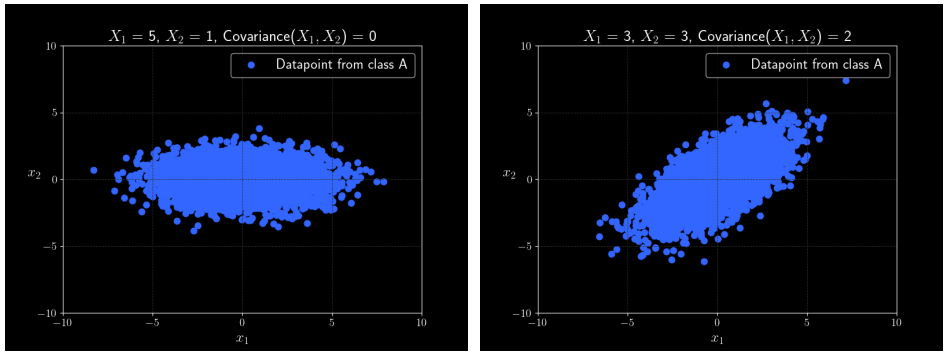


Figure 3: This figure shows the two datasets with datapoints in $\mathbb{R}^2$. On the left side the emperical covariance matrix is $\begin{bmatrix} 5 & 0 \\ 0 & 1 \end{bmatrix}$ and on the right side $\begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}$

LDA formular for $\mathbf{w}$ and β

given a dataset $X \in \mathbb{R}^{d\times n}$ and class means $\mathbf{x}_+, \mathbf{x}_- \in \mathbb{R}^d$

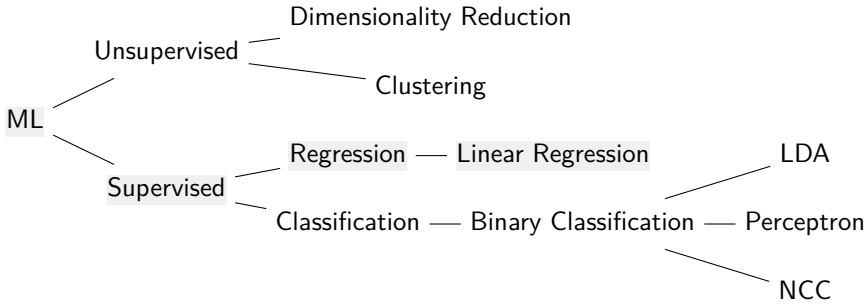
$$\mathbf{w} = {S_w}^{-1}(\mathbf{x}_+ - \mathbf{x}_-)$$
$$\beta = \frac{1}{2}\mathbf{w}^T(\mathbf{x}_+ + \mathbf{x}_-) + \log\left(\frac{n_-}{n_+}\right).$$

where S_w is

$$S_w = S_+ + S_- = \frac{1}{n_+}(X_+ - \bar{X}_+)(X_+ - \bar{X}_+)^\top + \frac{1}{n_-}(X_- - \bar{X}_-)(X_- - \bar{X}_-)^\top$$

and X_+ contains datapoints labeled $+$.

Regression



The goal of regression is to fit a function on a dataset. We want to predict the continous label of new points. For example measure datapoints and then fit a curve to smooth out. We want to predict the value of some underlying function.

Linear Regressioin with mapping on output

$$\hat{f} : \mathbb{R}^d \to \mathbb{R}, \quad x \mapsto \mathbf{w}^\top \varphi(x)$$

where φ is a function, weight vector $\mathbf{w} \in \mathbb{R}^m$, data point $x \in \mathbb{R}^d$, and bias $\beta \in \mathbb{R}$.

Example

If we choose $d = 1$ and $\varphi(x) = x$ we get the most simple form of linear regression:

$$\hat{f} : \mathbb{R} \to \mathbb{R}, \quad x \mapsto wx$$

where weight vector $w \in \mathbb{R}$, data point $x \in \mathbb{R}$ and $w^\top \varphi(x) = w^\top x \overset{d=1}{=} wx$.

A plot of this example can be seen on the right side of this page. An assumption for our model is that we assume that data follows some underlying function and that the data points are gaussianly distributed around it.

Simple Linear Regression

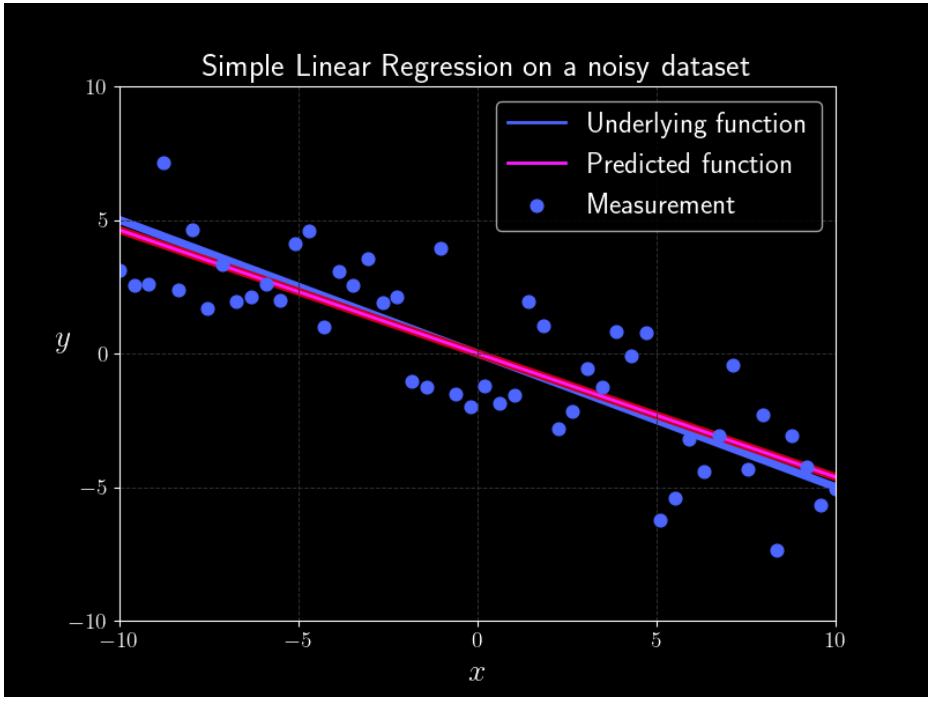


Figure 4: The blue dots (x, y) show one dimensional datapoints $x \in \mathbb{R}$ with their label $y \in \mathbb{R}$. These datapoints are sampled from an underlying function $f(x) = -0.5x$(blue line) with noise. The Predicted function $\hat{f}(x) = -0.49x$(pink line) is the result of simple linear regression and uses $w = 1/(XX^T)Xy^T \in \mathbb{R}$, with datamatrix $X \in \mathbb{R}^{1\times n}$, list of labels $\mathbb{R}^d$, for the weight vector.

We can change the shape of $\hat{f}$ by changing φ

Example

If we choose $d = 1$ and $\varphi(x) = \begin{bmatrix} 1 \\ x \end{bmatrix}$ we get simple linear regression with an offset:

$$\hat{f} : \mathbb{R} \to \mathbb{R}, \quad x \mapsto w_1 + w_2 x$$

where weight vector $w \in \mathbb{R}^2$, data point $x \in \mathbb{R}$
and $\mathbf{w}^\top \varphi(x) = \begin{bmatrix} w_1 & w_2 \end{bmatrix} \begin{bmatrix} 1 \\ x \end{bmatrix} = w_1 + w_2 x.$

Similiarly we can derive the shape of a polynomial, by using a polynomial mapping function φ

Example

$d = 1$, $\varphi(x) = \begin{bmatrix} 1 & x & x^2 & \dots & x^p \end{bmatrix}^\top$ then

$$\hat{f} : \mathbb{R} \to \mathbb{R}, \quad x \mapsto w_0 + w_1 x + w_2 x^2 + \dots w_p x^p$$

since

$$\mathbf{w}^\top \varphi(x) = \begin{bmatrix} w_0 & w_1 & w_2 & \dots & w_p \end{bmatrix} \begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^p \end{bmatrix} = w_0 + w_1 x + w_2 x^2 + \dots w_p x^p.$$

After choosing a mapping function we will need to feed our model training data to calculate the weights $\mathbf{w}$. How do we do that? We compare how much our predicted value $\hat{f}(x) = \hat{y}$ on our training datapoints deviate from the true values $\hat{y}$. For n datapoints with n labels we can use the Norm

$||\mathbf{y} - \mathbf{w}^\top \varphi(X)|| = ||\mathbf{y} - \hat{\mathbf{y}}|| = \sqrt{y_1 - \hat{y}_1^2 + y_2 - \hat{y}_2^2 + \dots y_n - \hat{y}_n^2}$ is proportional to $||y - \hat{y}||^2$

We use remove the square root function because it is expensive to compute. We define the ordinary least squares error with restriction of w as

Ordinary Least Squares with regularization

$$\mathcal{E}_{rr}(\mathbf{w}) := \left\|\mathbf{y} - \mathbf{w}^\top \varphi(X)\right\|^2 + \underbrace{\lambda \left\|\mathbf{w}\right\|^2}_{\text{regularization}} = \left(\sum_{i=1}^{N} (y_i - \mathbf{w}^\top \varphi(\mathbf{x}_i))^2\right) + \underbrace{\lambda(\mathbf{w}^\top \mathbf{w})}_{\text{regularization}}$$

For Ordinary Least Squares ($\mathcal{E}_{OLS}$) set $\lambda = 0$. From this formular we derive a $\mathbf{w}$ such that it is the minimum of the function $\mathcal{E}_{rr}$.

Minimum of Ordinary Least Squares with regularization

$$\mathbf{w} = (XX^\top + \lambda I)^{-1} X \mathbf{y}^\top$$

where $w \in \mathbb{R}^d$, $X \in \mathbb{R}^{n \times d}$ matrix containing data points and $I \in \mathbb{R}^{n \times n}$ is the Identity matrix.

Again to derive the solution for $\mathcal{E}_{OLS}$ set $\lambda = 0$.

Kernels

Mapping function φ

$$\varphi : \mathcal{X} \to \mathcal{F}, \quad \mathbf{x} \mapsto \varphi(\mathbf{x})$$

where $\mathcal{X}$ is the space where our datapoints come from (usually $\mathbb{R}^d$) and $\mathcal{F}$ called **feature space** is some other space (usually $\mathbb{R}^m$ with $m > d$).

Kernel function k corresponding to φ

$$k : \mathcal{X} \times \mathcal{X} \to \mathbb{R}, \quad k(\mathbf{x}_i, \mathbf{x}_j) \mapsto \varphi(\mathbf{x}_i)^\top \varphi(\mathbf{x}_j)$$

where $\mathcal{X}$ is the space where our datapoints come from (In this course usually $\mathcal{X} = \mathbb{R}^d$, d dimension of datapoint) and $\mathbf{x}_i, \mathbf{x}_j$ are the i-th and j-th datapoint. Functions that have these properties, are called **kernel functions** or **kernel**.

Gram matrix

We call the matrix defined as

$$K := \begin{bmatrix} k(\mathbf{x}_1, \mathbf{x}_1) & k(\mathbf{x}_1, \mathbf{x}_2) & \dots & k(\mathbf{x}_1, \mathbf{x}_n) \\ k(\mathbf{x}_2, \mathbf{x}_1) & k(\mathbf{x}_2, \mathbf{x}_2) & \dots & k(\mathbf{x}_2, \mathbf{x}_n) \\ \vdots & \vdots & \ddots & \vdots \\ k(\mathbf{x}_n, \mathbf{x}_1) & k(\mathbf{x}_n, \mathbf{x}_2) & \dots & k(\mathbf{x}_n, \mathbf{x}_n) \end{bmatrix} = k(X_{train}, X_{train}) \in \mathbb{R}^{n \times n}$$

the **Gram matrix**. In each entry $K_{[i,j]}$ we have the result of our kernel function on the i-th and j-th datapoint. This matrix is symmetric positive semi definite.

Kernel Trick and Mercer's Theorem

Kernel trick: if $\mathbf{x}$ enters only in scalar products, then we can replace each scalar product with $k(\mathbf{x}_i, \mathbf{x}_j)$.

$$\begin{tikzcd} \varphi(\mathbf{x}_i)^\top \varphi(\mathbf{x}_j) \arrow[r, bend left, "\text{Kernel Trick}"] \arrow[r, bend right, "\text{Mercer's Theorem}"'] & = & k(\mathbf{x}_i, \mathbf{x}_j) \end{tikzcd}$$

Mercer's theorem: If k is a valid kernel function one can construct a feature space and a mapping φ such that the scalar product in the mapped space is the same as k

Gaussian Kernel

$x_1, x_2 \in \mathbb{R}^d$ Datapoints and $\sigma \in \mathbb{R}$, then

$$\text{rbf} : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}, \quad (x_1, x_2) \mapsto e^{-\frac{\|x_1 - x_2\|^2}{2\sigma^2}}$$

defines the gaussian kernel.

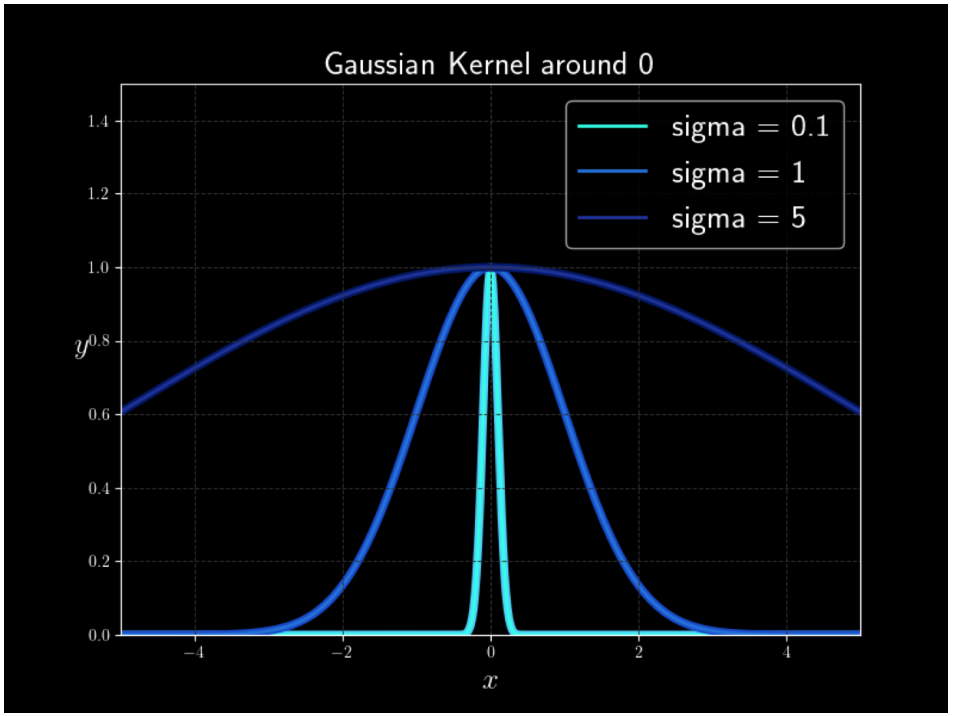


Figure 2: The plot shows the one dimensional case of the function rbf(x_1, x_2) with $x_1 = 0$. Darker shades of blue denote higher values for the hyperparameter σ. When x_2 values are close to $x_1 = 0$, then rbf returns a higher value than x_2 values with a larger distance.

Application of Kernels

Kernel Ridge Regression

instead of $\mathbf{w}$ we **calculate** $\alpha \in \mathbb{R}^n$

$$\alpha = (K + \lambda I)^{-1}\mathbf{y}^\top$$

where K is the gram matrix of k, $\lambda \in \mathbb{R}$ is our regularization parameter, I is the identity matrix and $\mathbf{y}$ are the labels of our datapoints. We then **predict** new datapoints with

$$f(x_{new}) = y = \sum_{i=1}^{n} \alpha_i k(x_i, x_{new})$$

Unsupervised Learning

Math Recap

Eigenvalues; Eigenvectors

$$Av = \lambda v$$

A Matrix, v eigenvector corresponding to eigenvalue λ.

Principal Component Analysis

The goal of PCA is to maximize the variance of projected data.
The variance of data projected onto $\mathbf{w}$ which can be rewritten as

$$\overline{Var}(\mathbf{w}^\top X) = \mathbf{w}^\top \Sigma_x \mathbf{w}$$

Finding $\mathbf{w}$ that maximizes that is equivalent to the $\mathbf{w}$ that maximizes

$$\frac{\mathbf{w}^\top X X^\top \mathbf{w}}{\mathbf{w}^\top \mathbf{w}}$$

Deriving

$$\frac{\partial}{\partial \mathbf{w}} \frac{\mathbf{w}^\top X X^\top \mathbf{w}}{\mathbf{w}^\top \mathbf{w}} = \frac{\mathbf{w}^\top \mathbf{w} 2 S \mathbf{w} - \mathbf{w} \top S \mathbf{w} 2 \mathbf{w}}{(\mathbf{w}^\top \mathbf{w})^2}$$

and setting this to zero leads to

$$\underbrace{XX^\top}_{S} \mathbf{w} = \underbrace{\frac{\mathbf{w} S \mathbf{w}}{\mathbf{w}^\top \mathbf{w}}}_{\in \mathbb{R}} \mathbf{w}.$$

Which means that the eigenvectors maximize the variance. From this we can derive an algorithm to reduce dimensions.

Principal Component Analysis (PCA)

1. Compute empirical covariance matrix $\Sigma_x \in \mathbb{R}^{d \times d}$
2. Compute eigenvalues of Σ_x
3. Compute for k largest eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_k \in \mathbb{R}$ calculate corresponding eigenvectors $w_1, w_2, \ldots, w_k \in \mathbb{R}^d$. The do
$$W = \begin{bmatrix} | & | & \ldots & | \\ w_1 & w_2 & \ldots & w_k \\ | & | & \ldots & | \end{bmatrix}.$$
4. Project $H = W^\top X$
5. For reconstruction you can $X \approx W \cdot H$

Linear Kernel PCA

1. Compute $K = X^\top X \in \mathbb{R}^{n \times n}$, X is centered.
2. Compute eigenvalues of K
3. Compute for k largest eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_k \in \mathbb{R}$ calculate corresponding eigenvectors $v_1, v_2, \ldots, v_k \in \mathbb{R}^n$. The do
$$\alpha = \begin{bmatrix} | & | & \ldots & | \\ v_1 & v_2 & \ldots & v_k \\ | & | & \ldots & | \text{ and then} \end{bmatrix} W = X\alpha.$$
4. Project $H = W^\top X$
5. For reconstruction you can $X \approx W \cdot H$

K-Means Clustering

The goal of k-means clustering is to minimize

$$\sum_{i=1}^{n} \|x_i - \mu_{r_i}\|^2$$

k-Means Clustering

Let $X \in \mathbb{R}^{n \times d}$ datamatrix containing n datapoints $\mathbf{x} \in \mathbb{R}^d$ then

1. Initiate k vectors $c \in \mathbb{R}^n$ (randomly) then

$$C = \begin{bmatrix} | & | & \dots & | \\ c_1 & c_2 & \dots & c_k \\ | & | & \dots & | \end{bmatrix} \in \mathbb{R}^{n \times k}.$$

   This is the cluster matrix, it contains clusterpoints. (each column is a clusterpoint, clustervector)

2. For each datapoint $\mathbf{x}_i$ assign to clusterpoint based on distance.

$$\text{closest} := \begin{bmatrix} | & | & \dots & | \\ y_1 & y_2 & \dots & y_k \\ | & | & \dots & | \end{bmatrix} \in \{0,1\}^{n \times k}.$$

3. Calculate mean of set of datapoints belonging to the same cluster. This is new clusterpoint.

4. Repeat 1) 2) until maximum number of iterations is reached or clusterpoints only change by small value $\varepsilon > 0 \in \mathbb{R}$

Nonnegative Matrix Factorization

Objective of NMF is

$$\text{argmin}_{W,H} \sum_{i=1}^{d} \sum_{j=1}^{n} (X_{ij} - (WH)_{ij})^2$$

Cognitive Algorithms Tutorial Notes Hannah Louisa Boldt
Notes are not complete and can contain typos. If you spot any mistakes, typos or have any questions please email me at `h.boldt@campus.tu-berlin.de`*!*